Jetson Sensor Hub: A Virtual Sensor Telemetry Platform for Real-Time Environmental Monitoring

Om Patel (opate@uic.edu)
Michael E. Papka (papka@uic.edu)
University of Illinois Chicago

SAGE sagecontinuum.org

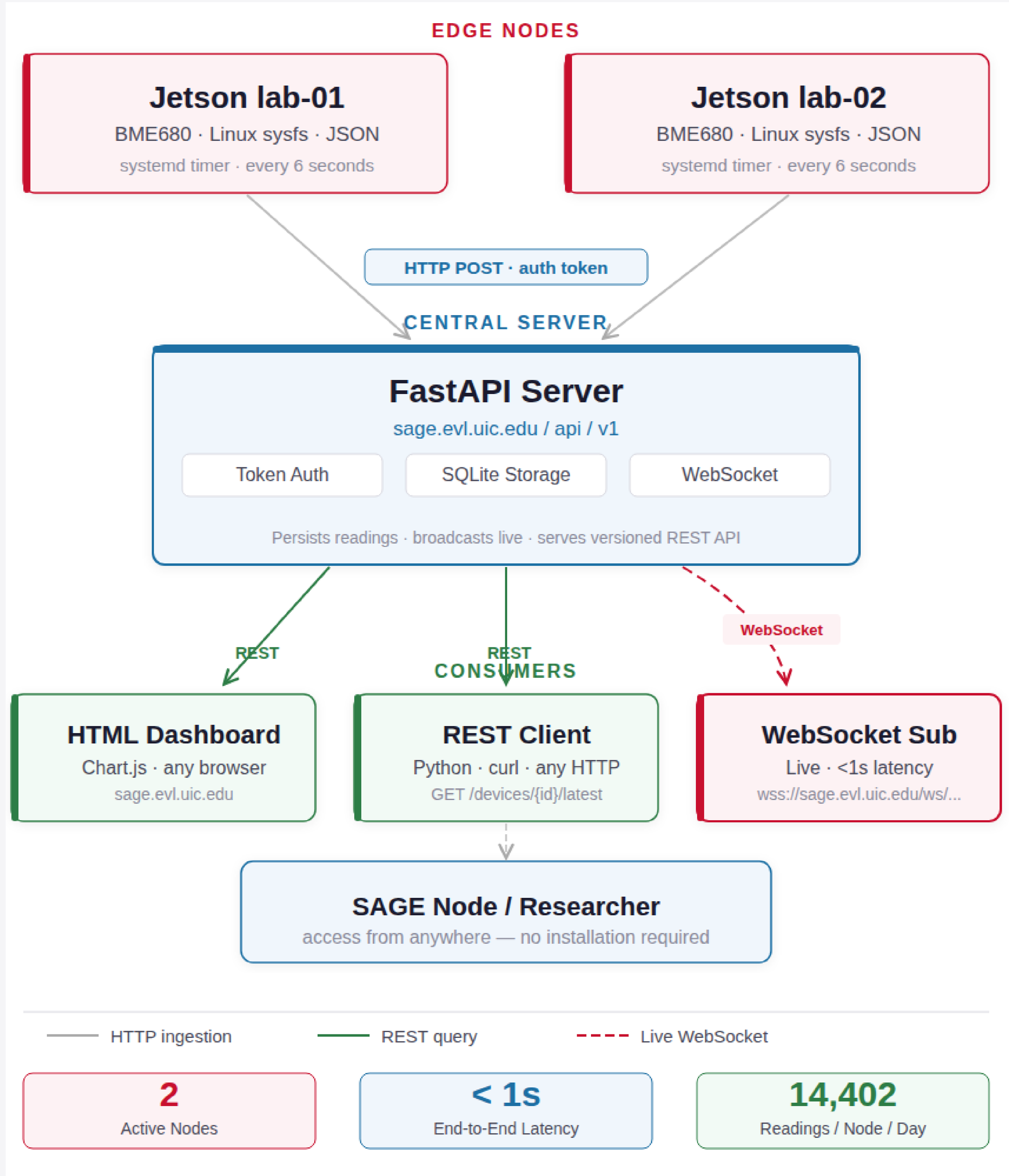
Motivation & Problem

- Large-scale sensor networks like the NSF SAGE testbed need lightweight infrastructure to share environmental data across distributed edge nodes without cloud brokers or heavy infrastructure.
- Runs entirely on the edge node
- Exposes REST + real-time WebSocket endpoints
- Persists a queryable, exportable time-series history
- Zero cloud dependency — publicly accessible

Current Deployment

- **jetson-lab-01 — Server Room** Monitors ambient conditions in the EVL server room, temperature, humidity, and pressure — to detect anomalies that could affect critical infrastructure.
- **jetson-lab-02 — Lab Space, next to NVIDIA Thor** Tracks thermal output of an NVIDIA Jetson AGX Thor during active experiments. On May 1, 2026, a spike to **40.39°C** was captured and broadcast live via WebSocket. Ambient baseline is ~29°C.

System Architecture



Key Findings

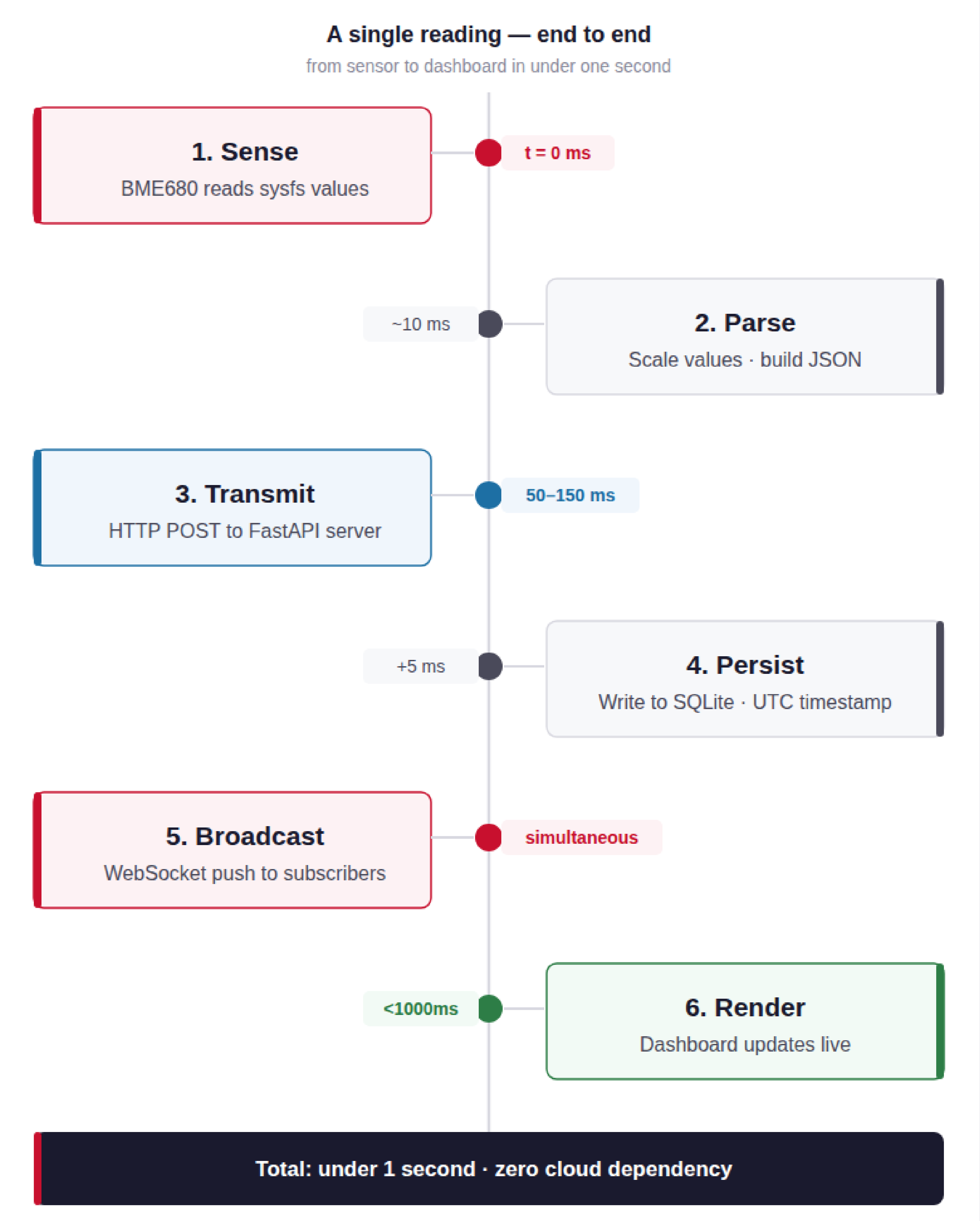
- **Virtual sensor on constrained hardware** — A Jetson simultaneously handles acquisition, ingestion, SQLite persistence, and live WebSocket broadcast — all under 1 second end-to-end.
- **Zero cloud dependency** — The API is globally queryable via sage.evl.uic.edu with a single HTTP or WebSocket call — no broker, no external account.
- **Decoupled producer and consumer** — Producers push at a fixed 6s cadence; consumers independently poll, stream live, or batch-export full days as CSV.
- **Static HTML frontend** — A single Chart.js file serves the entire dashboard — no Python runtime, no build step, accessible from any browser.
- **Token auth balances openness** — Per-device tokens secure ingestion while all read endpoints stay fully public for researchers and SAGE nodes.

Live Dashboard

Limitations

- **Single point of failure** — The central FastAPI server is a bottleneck; if it goes down, all ingestion and streaming stops until restarted.
- **No WebSocket replay** — Subscribers only receive live readings. Data missed during a disconnection cannot be retrieved via the WebSocket path.
- **Fixed ingestion cadence** — The 6-second push interval is set at the agent level; the server cannot request data on-demand or at variable rates.
- **Not load-tested at scale** — Currently deployed on 2 nodes. The WebSocket broadcaster has not been tested under high concurrent subscriber load.

Data Flow



Future Work

- **Serverless virtual sensor** — Remove the central server entirely. Each Jetson runs its own WebSocket endpoint and subscribers connect directly — no hub, no persistence layer, no broker. Eliminates the single point of failure and scales horizontally as nodes are added.
- **Virtual camera as SAGE plugin** — Extend the same model to camera streams. Each Jetson captures MJPEG video and exposes it as a SAGE plugin endpoint. Any node on the testbed subscribes to a live feed without a direct connection to the source — the same publish-subscribe pattern as the sensor system, applied to video.
- **Open challenges** — NAT traversal for Jetsons deployed in wild, distributed node discovery across sensor and camera streams, adaptive quality for MJPEG bandwidth constraints, and multi-node federation for a unified view across the SAGE testbed.

References & Acknowledgements

[1] P. Beckman et al., "SAGE: A Software-Defined Sensor Network," IEEE IPDPS Workshops, 2020.
[2] Bosch Sensortec, "BME680 Datasheet," 2017.
[3] Linux Kernel, "IIO Subsystem," kernel.org, 2024.
[4] NVIDIA, "Jetson AGX Orin Developer Kit," 2022.
[5] Chart.js Contributors, chartjs.org, 2024.

This work was supported in part by the National Science Foundation under Awards No. 2331263, and 2436842.